01. Torrust Tracker 전체 아키텍처 분석

1.1 패키지 구조 개요

Torrust Tracker는 Rust 워크스페이스 기반의 모듈러 아키텍처를 채택하고 있습니다. 전체 시스템은 23개의 독립적인 패키지로 구성되어 있으며, 각 패키지는 명확한 책임과 역할을 가지고 있습니다.

워크스페이스 구조

메인 애플리케이션:

- `torrust-tracker` (루트): 메인 실행 파일과 애플리케이션 진입점

핵심 패키지 (23개):

```
packages/
├── tracker-core/ # 핵심 비즈니스 로직
├── http-tracker-core/ # HTTP 트래커 핵심 로직
├── udp-tracker-core/ # UDP 트래커 핵심 로직
├── rest-tracker-api-core/ # REST API 핵심 로직
├── configuration/ # 설정 관리
├── events/ # 이벤트 시스템
├── metrics/ # 메트릭 수집
├── clock/ # 시간 관리
├── primitives/ # 기본 데이터 타입
├── located-error/ # 에러 처리
```

```
├── swarm-coordination-registry/ # 스웜 조정 레지스트리
├── axum-http-tracker-server/ # HTTP 서버 구현
├── axum-rest-tracker-api-server/ # REST API 서버 구현
├── axum-health-check-api-server/ # 헬스체크 API 서버
├── axum-server/ # Axum 서버 공통 기능
├── udp-tracker-server/ # UDP 서버 구현
├── server-lib/ # 서버 공통 라이브러리
├── http-protocol/ # HTTP 프로토콜 구현
├── udp-protocol/ # UDP 프로토콜 구현
├── tracker-client/ # 트래커 클라이언트
├── rest-tracker-api-client/ # REST API 클라이언트
├── test-helpers/ # 테스트 유틸리티
└── torrent-repository-benchmarking/ # 성능 벤치마킹
```

의존성 관계 분석

메인 애플리케이션 의존성 (`Cargo.toml`):

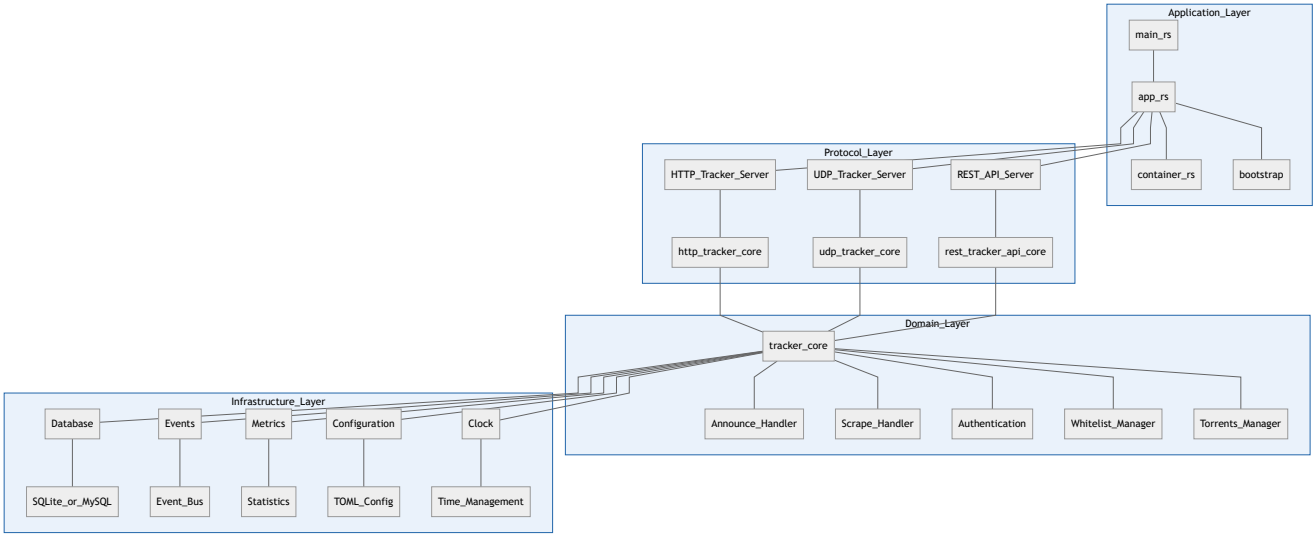
```
[dependencies]
# 핵심 트래커 로직
bittorrent-tracker-core = { path = "packages/tracker-core" }
bittorrent-http-tracker-core = { path = "packages/http-tracker-core" }
bittorrent-udp-tracker-core = { path = "packages/udp-tracker-core" }
```

```
# 서버 구현체

torrust-axum-http-tracker-server = { path = "packages/axum-http-tracker-server"

torrust-udp-tracker-server = { path = "packages/udp-tracker-server" }

torrust-axum-rest-tracker-api-server = { path = "packages/axum-rest-tracker-api-



# 공통 라이브러리

torrust-tracker-configuration = { path = "packages/configuration" }

torrust-server-lib = { path = "packages/server-lib" }

torrust-rest-tracker-api-core = { path = "packages/rest-tracker-api-core" }
```

1.2 레이어 아키텍처 분석

Torrust Tracker는 명확한 레이어 분리를 통해 관심사의 분리(Separation of Concerns)를 구현하고 있습니다.

아키텍처 레이어 구조



1. Application Layer (애플리케이션 레이어)

역할: 애플리케이션의 진입점과 전체 시스템 조정

주요 컴포넌트:

- `src/main.rs` : 애플리케이션 진입점
- `src/app.rs` : 애플리케이션 라이프사이클 관리
- `src/container.rs` : 의존성 주입 컨테이너
- `src/bootstrap/` : 초기화 로직

함수 예시:

```
// src/main.rs
#[tokio::main]
async fn main() {
let (_app_container, jobs) = app::run().await;
tokio::select! {
_ = tokio::signal::ctrl_c() => {
tracing::info!("Torrust tracker shutting down ...");
jobs.cancel();
jobs.wait_for_all(Duration::from_secs(10)).await;
}
}
}

// src/app.rs
pub async fn run() -> (Arc<AppContainer>, JobManager) {
let (config, app_container) = bootstrap::app::setup();
let app_container = Arc::new(app_container);
```

```
let  jobs = start(&config, &app_container).await;

(app_container, jobs)

}
```

2. Protocol Layer (프로토콜 레이어)

역할: 네트워크 프로토콜별 구현과 요청/응답 처리

주요 컴포넌트:

- `packages/axum-http-tracker-server/` : HTTP 트래커 서버
- `packages/udp-tracker-server/` : UDP 트래커 서버
- `packages/axum-rest-tracker-api-server/` : REST API 서버
- `packages/http-protocol/` : HTTP 프로토콜 구현
- `packages/udp-protocol/` : UDP 프로토콜 구현

특징:

- 각 프로토콜별로 독립적인 서버 구현
- 공통 코어 로직에 대한 의존성
- Axum 웹 프레임워크 기반 HTTP 구현

3. Domain Layer (도메인 레이어)

역할: BitTorrent 트래커의 핵심 비즈니스 로직

주요 컴포넌트:

- `packages/tracker-core/` : 핵심 트래커 로직
- Announce Handler: 피어 등록 및 관리
- Scrape Handler: 토렌트 통계 처리

- Authentication: 인증 키 관리
- Whitelist Manager: 허용 토렌트 관리
- Torrents Manager: 토렌트 생명주기 관리

핵심 서비스:

```
// tracker-core의 주요 컴포넌트들
pub struct TrackerCoreContainer {
pub announce_handler: Arc<AnnounceHandler>,
pub scrape_handler: Arc<ScrapeHandler>,
pub keys_handler: Arc<KeysHandler>,
pub whitelist_manager: Arc<WhitelistManager>,
pub torrents_manager: Arc<TorrentsManager>,
// ...
}
```

4. Infrastructure Layer (인프라 레이어)

역할: 외부 시스템과의 연동 및 공통 기능 제공

주요 컴포넌트:

- `packages/configuration/` : 설정 관리 (TOML 기반)
- `packages/events/` : 이벤트 시스템
- `packages/metrics/` : 메트릭 수집 및 통계
- `packages/clock/` : 시간 관리
- `packages/primitives/` : 기본 데이터 타입

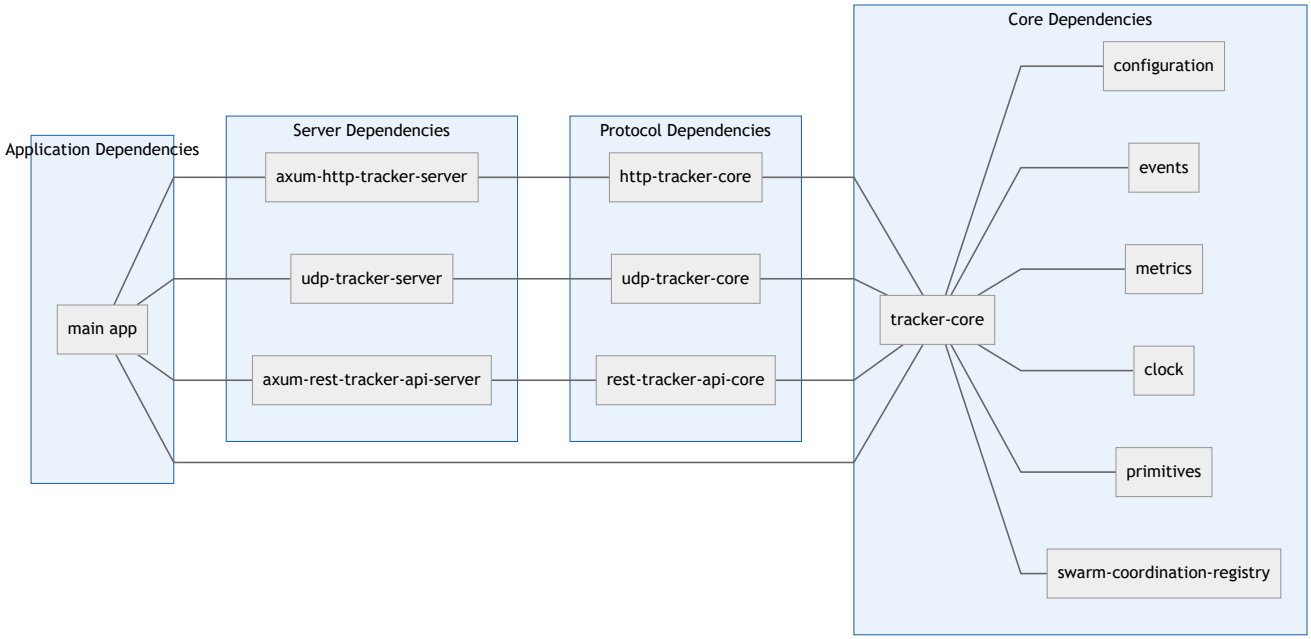
- `packages/located-error/` : 에러 처리

데이터베이스 지원:

- SQLite (기본)
- MySQL (선택적)
- R2D2 연결 풀 사용

1.3 의존성 관계 분석

핵심 의존성 흐름



의존성 계층 구조

Level 1 - 기반 라이브러리:

- `primitives` : 기본 데이터 타입
- `located-error` : 에러 처리
- `clock` : 시간 관리

Level 2 - 공통 서비스:

- `configuration` : 설정 관리
- `events` : 이벤트 시스템
- `metrics` : 메트릭 수집

Level 3 - 도메인 로직:

- `tracker-core` : 핵심 비즈니스 로직
- `swarm-coordination-registry` : 스웜 조정

Level 4 - 프로토콜 구현:

- `http-tracker-core` : HTTP 트래커 로직
- `udp-tracker-core` : UDP 트래커 로직
- `rest-tracker-api-core` : REST API 로직

Level 5 - 서버 구현:

- `axum-http-tracker-server` : HTTP 서버
- `udp-tracker-server` : UDP 서버
- `axum-rest-tracker-api-server` : REST API 서버

Level 6 - 애플리케이션:

- `torrust-tracker` : 메인 애플리케이션

1.4 핵심 컴포넌트 역할

AppContainer - 의존성 주입 컨테이너

위치: `src/container.rs`

역할: 전체 애플리케이션의 의존성 관리 및 컴포넌트 생명주기 관리

주요 기능:

```
pub struct AppContainer {
// 설정
pub http_api_config: Arc<Option<HttpApi>>,
// 레지스트라
pub registar: Arc<Registar>,
// 스웜 조정 레지스트리
pub swarm_coordination_registry_container: Arc<SwarmCoordinationRegistryContain
// 핵심 트래커
pub tracker_core_container: Arc<TrackerCoreContainer>,
// HTTP 트래커
pub http_tracker_core_services: Arc<HttpTrackerCoreServices>,
pub http_tracker_instance_containers: Arc<HashMap<SocketAddr, Arc<HttpTrackerCo
// UDP 트래커
pub udp_tracker_core_services: Arc<UdpTrackerCoreServices>,
pub udp_tracker_server_container: Arc<UdpTrackerServerContainer>,
pub udp_tracker_instance_containers: Arc<HashMap<SocketAddr, Arc<UdpTrackerCore
```

```
}
```

초기화 과정:

```
impl AppContainer {
pub fn initialize(configuration: &Configuration) -> AppContainer {
// 1. 설정 초기화
let core_config = Arc::new(configuration.core.clone());
// 2. 레지스트라 초기화
let registar = Arc::new(Registar::default());
// 3. 스웜 조정 레지스트리 초기화
let swarm_coordination_registry_container = Arc::new(
SwarmCoordinationRegistryContainer::initialize(
core_config.tracker_usage_statistics.into()
)
);
// 4. 트래커 코어 초기화
let tracker_core_container = Arc::new(
TrackerCoreContainer::initialize_from(
&core_config,
&swarm_coordination_registry_container
)
);
// 5. HTTP/UDP 서비스 초기화
```

```
// ...
}
}
```

JobManager - 작업 관리자

위치: `src/bootstrap/jobs/manager.rs`

역할: 백그라운드 작업들의 생명주기 관리

주요 기능:

```
pub struct JobManager {
jobs: Vec<Job>,
cancellation_token: CancellationToken,
}

impl JobManager {
pub fn push<N: Into<String>>(&mut self, name: N, handle: JoinHandle<()>) {
self.jobs.push(Job::new(name, handle));
}
pub fn cancel(&self) {
self.cancellation_token.cancel();
}
pub async fn wait_for_all(mut self, grace_period: Duration) {
// 모든 작업이 완료될 때까지 대기
```

```
}

}
```

서비스 시작 과정

위치: `src/app.rs`

주요 시작 단계:

```
pub  async  fn  start(config: &Configuration, app_container: &Arc<AppContainer>)

// 1. 서비스 활성화 경고

warn_if_no_services_enabled(config);

// 2. 데이터베이스에서 데이터 로드

load_data_from_database(config, app_container).await;

// 3. 작업들 시작

start_jobs(config, app_container).await

}


async  fn  start_jobs(config: &Configuration, app_container: &Arc<AppContainer>)

let  mut  job_manager = JobManager::new();

// 이벤트 리스너들 시작

start_swarm_coordination_registry_event_listener(config, app_container, &mut  jo

start_tracker_core_event_listener(config, app_container, &mut  job_manager);

// 서버 인스턴스들 시작

start_the_udp_instances(config, app_container, &mut  job_manager).await;
```

```
start_the_http_instances(config, app_container, &mut  job_manager).await;

// 백그라운드 작업들 시작

start_torrent_cleanup(config, app_container, &mut  job_manager);

start_peers_inactivity_update(config, app_container, &mut  job_manager);

// API 서비스들 시작

start_the_http_api(config, app_container, &mut  job_manager).await;

start_health_check_api(config, app_container, &mut  job_manager).await;

job_manager

}
```

설정 관리 시스템

위치: `packages/configuration/`

특징:

- TOML 기반 설정 파일
- 환경 변수 지원 (`TORRUST_TRACKER_CONFIG_TOML`)
- 런타임 검증
- 비밀 정보 마스킹

설정 구조:

```
[logging]

threshold = "info"
```

```
[core]

inactive_peer_cleanup_interval = 600

listed = false

private = false

tracker_usage_statistics = true



[core.database]

driver = "sqlite3"

path = "./storage/tracker/lib/database/sqlite3.db"



[[http_trackers]]

bind_address = "0.0.0.0:7070"



[[udp_trackers]]

bind_address = "0.0.0.0:6969"



[http_api]

bind_address = "127.0.0.1:1212"
```

결론

Torrust Tracker는 잘 설계된 모듈러 아키텍처를 통해 다음과 같은 이점을 제공합니다:

1. **명확한 책임 분리**: 각 패키지가 특정 역할을 담당

2. **확장성**: 새로운 프로토콜이나 기능 추가가 용이

3. **테스트 용이성**: 각 컴포넌트의 독립적 테스트 가능

4. **유지보수성**: 레이어별 분리로 변경 영향 최소화

5. **성능**: 비동기 처리와 효율적인 리소스 관리

이러한 아키텍처는 BitTorrent 트래커의 복잡한 요구사항을 체계적으로 해결하면서도 코드의 가독성과 유지보수성을 보장합니다.